



人工智能基础实验报告

题目编号	5C 轻量级实时目标检测优化 工业质检场景
------	-----------------------

题 目	基于 YOLOv8n 的钢材表面缺陷检测优化实验
--------	--------------------------

小组成员	刘紫涵	2024141450238
------	-----	---------------

	刘宇欣	2024141450237
--	-----	---------------

	王 芯	2024141450214
--	------	---------------

	李 钰	2024141450160
--	------	---------------

	余玉媛	2024141450354
--	-----	---------------

目 录

摘要	1
1 引言	2
1.1 研究背景	2
1.2 问题与挑战	2
1.3 本项目主要工作	3
2 方法	3
2.1 数据集与预处理	3
2.2 基线检测模型: YOLOv8n	6
2.3 知识蒸馏方法	9
2.4 结构化通道剪枝方法	12
2.5 评价指标	16
3 实验与结果分析	17
3.1 实验环境与统一设置	17
3.2 Baseline 实验结果与分辨率影响分析	18
3.3 训练收敛性分析	21
3.4 知识蒸馏实验结果	22
3.5 剪枝实验结果	25
3.6 全部模型综合对比	29
3.7 精度—速度权衡分析	31
3.8 混淆矩阵与类别误差分析	32
3.9 检测可视化与误检漏检案例分析	33
4 讨论	34
4.1 不同压缩方法的优缺点	34
4.2 输入分辨率对部署的影响	35
4.3 实际部署限制	35
5 结论	35
6 分工声明	36

摘要

随着深度学习目标检测模型在工业质检领域的广泛应用, 钢材表面缺陷检测逐渐成为智能制造中的重要任务。传统人工检测方式存在效率低、主观性强、难以持续稳定工作等问题, 而基于深度学习的目标检测方法能够自动识别缺陷类别并定位缺陷区域, 在提升检测效率和产品质量控制方面具有重要意义。然而, 目标检测模型通常存在参数量较大、计算量较高、推理速度受限等问题, 难以直接部署到边缘设备或算力受限的工业现场终端。因此, 如何在保证检测精度的前提下降低模型复杂度、提升推理速度, 是轻量级实时目标检测优化中的关键问题。

本项目面向工业质检场景中的钢材表面缺陷检测任务, 采用 NEU-DET 钢材表面缺陷数据集作为实验数据集, 选择 YOLOv8n 作为基线检测模型。围绕轻量化实时目标检测优化目标, 本文主要实现并比较了知识蒸馏和结构化剪枝两种模型压缩方法。其中, 知识蒸馏通过教师模型引导学生模型学习更有效的检测特征, 旨在提升轻量模型的检测性能; 结构化剪枝则通过移除不重要的通道来降低模型参数量和计算量, 并结合微调训练恢复模型精度。实验同时在 416×416 和 640×640 两种输入分辨率下开展, 以分析输入分辨率对检测精度、计算复杂度和推理速度的影响。

实验结果表明, YOLOv8n 基线模型能够在 NEU-DET 数据集上取得较好的检测效果; 提高输入分辨率有助于增强模型对弱纹理、小目标缺陷的识别能力, 但也会带来更高的计算量和更低的推理速度。知识蒸馏在不改变学生模型参数量的前提下, 可在部分设置下保持或略微改善检测性能, 但其效果受教师模型性能、输入分辨率和蒸馏策略影响; 结构化剪枝能够明显减少模型参数量和计算量, 但较高剪枝比例可能导致检测精度下降, 因此剪枝后微调是恢复模型性能的重要步骤。综合来看, 模型压缩方法能够在精度、模型复杂度和实时性之间取得一定平衡, 为工业缺陷检测模型在边缘设备上的部署提供了可行思路。

关键词: 工业缺陷检测; YOLOv8n; NEU-DET; 知识蒸馏; 模型剪枝; 轻量化目标检测

1 引言

1.1 研究背景

工业质检是智能制造体系中的重要环节。对于钢材生产过程而言，表面缺陷会直接影响产品质量、使用寿命和后续加工性能，因此需要在生产过程中及时发现并定位裂纹、夹杂物、斑块、麻面、轧入氧化皮和划痕等缺陷。传统人工检测方式依赖质检人员经验，不仅检测效率有限，而且容易受到疲劳、环境和主观判断的影响，难以满足高速生产线中实时、稳定、自动化的检测需求。

近年来，卷积神经网络和目标检测算法在图像识别、目标定位和工业视觉检测中取得了显著进展。YOLO 系列模型由于具有端到端检测、推理速度快和部署方便等特点，被广泛应用于实时目标检测任务。对于钢材表面缺陷检测而言，YOLO 模型能够同时完成缺陷位置定位和类别识别，适合构建自动化工业质检系统。

1.2 问题与挑战

然而，在实际工业部署场景中，仅有较高检测精度并不足够。工业现场常常需要将检测模型部署到 CPU、边缘计算盒、嵌入式设备或其他算力有限的平台上，这对模型的参数量、计算量和推理速度提出了更高要求。虽然较大的目标检测模型通常具有更强的特征提取能力，但其计算开销较高，可能难以满足实时检测需求。因此，如何在尽量保持检测精度的同时降低模型复杂度，是本项目关注的核心问题。

本项目选择 YOLOv8n 作为基线模型，围绕“轻量级实时目标检测优化”任务展开研究。YOLOv8n 是 YOLOv8 系列中较轻量的版本，具有较小的参数量和较快的推理速度，适合作为模型压缩和部署优化的基础模型。在此基础上，本文进一步引入知识蒸馏和结构化剪枝两种模型压缩方法：知识蒸馏侧重于利用教师模型提升学生模型的检测能力，结构化剪枝侧重于减少模型冗余通道以降低参数量和计算量。

此外，输入图像分辨率也是影响检测效果和推理效率的重要因素。较高分辨率通常能够保留更多细节信息，有利于识别小目标或弱纹理缺陷，但也会显著增

加模型计算量；较低分辨率能够提升推理速度，却可能造成部分细节缺失。因此，本文在 416×416 和 640×640 两种输入分辨率下进行实验对比，从检测精度、参数量、计算量和推理速度等方面综合评估不同模型配置的表现。

1.3 本项目主要工作

本文主要工作包括以下几个方面：第一，基于 NEU-DET 钢材表面缺陷数据集完成数据预处理与 YOLO 格式适配；第二，训练 YOLOv8n 基线模型，并比较不同输入分辨率下的检测效果；第三，实现知识蒸馏方法，分析其对轻量模型检测能力的提升作用；第四，实现结构化剪枝方法，并比较不同剪枝比例下模型复杂度与检测精度的变化；第五，通过综合对比表、精度-速度权衡曲线、混淆矩阵和可视化检测结果，对模型压缩效果进行系统分析。

2 方法

2.1 数据集与预处理

2.1.1 数据集来源

本实验面向工业质检场景中的钢材表面缺陷检测任务，采用 NEU-DET 钢材表面缺陷数据集作为实验数据集。该数据集由东北大学 (Northeastern University) 宋克臣团队制作并公开发布，是工业表面缺陷检测领域中较常用的标准数据集之一。NEU-DET 数据集来源于真实热轧钢带生产场景，图像中的缺陷形态、背景纹理和灰度变化均具有较强的工业实际特征，适合用于验证轻量级目标检测模型在工业质检任务中的检测能力。

NEU-DET 数据集共包含 1800 张钢材表面缺陷图像，原始图像均为单通道灰度图像，分辨率统一为 200×200 像素。数据集涵盖 6 类典型钢材表面缺陷，每类包含 300 张图像，类别分布较为均衡。均衡的数据分布能够减少类别不平衡对模型训练和评估结果造成的影响，使不同缺陷类别之间的检测性能具有更好的可比性。

2.1.2 缺陷类别

本实验涉及的 6 类缺陷分别为 crazing、inclusion、patches、pitted_surface、rolled-in_scale 和 scratches。各类别对应的中文名称及主要特征如表 1 所示。

表 1 NEU-DET 数据集缺陷类别说明

YOLO 类别 ID	英文名称	中文名称	缺陷特征与成因
0	Crazing	网状裂纹	表面呈细碎网状开裂，多由轧制过程中应力分布不均或冷却工艺不当导致。
1	Inclusion	夹杂物	钢材内部非金属杂质暴露于表面形成的点状或条状异物，与基体存在一定灰度差异。
2	Patches	氧化斑块	表面出现不规则片状色差区域，通常与氧化皮压入或表面成分分布不均有关。
3	Pitted Surface	麻面坑蚀	表面密集分布细小凹坑，可能由表面氧化层脱落或硬质异物压入形成。
4	Rolled-in Scale	轧入氧化皮	轧制过程中氧化铁皮被压入钢材表面，形态多为不规则片状或块状。
5	Scratches	机械划痕	沿轧制方向分布的长条状擦痕，多由生产线导辊或转运设备摩擦导致。

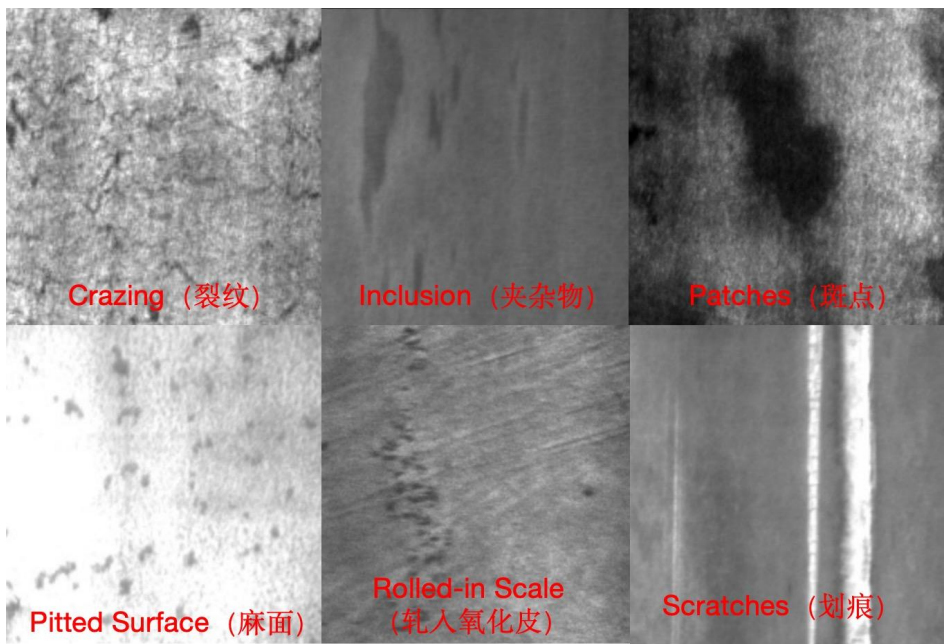


图 1 NEU-DET 数据集六类缺陷样本示例

从图 1 可以看出，不同类别缺陷在形态和纹理上存在明显差异。例如，scratches 通常表现为沿固定方向延伸的细长划痕，pitted_surface 多表现为密集分布的小凹坑，patches 则呈现较大面积的不规则斑块。相比之下，crazing 和 rolled-in_scale 的边界相对模糊，部分区域与钢材背景纹理较为接近，因此在后续检测任务中更容易出现漏检或定位不准确的问题。

2.1.3 数据标注格式转换

在数据标注方面，原始 NEU-DET 数据集采用 PASCAL VOC 格式进行标注，每张图像对应一个 .xml 标注文件。该文件记录了缺陷目标的类别名称以及边界框在原图中的绝对像素坐标，即 xmin、ymin、xmax 和 ymax。由于本项目采用 YOLO 系列目标检测模型进行训练和评估，因此需要将原始 PASCAL VOC 格式标注转换为 YOLO 模型所需的文本格式。

YOLO 格式标签以 .txt 文件保存，每个目标占一行，格式为：

```
class_id center_x center_y width height
```

其中，class_id 表示缺陷类别编号，center_x 和 center_y 表示边界框中心点相对于图像宽度和高度的归一化坐标，width 和 height 表示边界框宽度和高度相对于图像尺寸的归一化比例。通过这种格式转换，模型可以直接读取图像及其对应标注，从而完成监督式目标检测训练。

在数据目录组织方面，本实验按照 YOLO 官方推荐的方式对数据集进行整理，将图像文件和标签文件分别存放在 images/ 和 labels/ 子目录下，并通过 data.yaml 文件定义训练集路径、验证集路径、类别数量和类别名称。该组织方式能够保证训练脚本自动完成图像与标签的匹配读取，提高实验流程的规范性和可复现性。

2.1.4 数据集划分

本实验将 NEU-DET 数据集划分为训练集和验证集。其中，训练集共包含 1770 张图像，每类缺陷选取第 6 张至第 300 张图像，即每类 295 张；验证集共包含 30 张图像，每类缺陷选取前 5 张图像，即每类 5 张。该划分方式保证训练集与验证集之间不存在图像重叠，同时各类别在训练集和验证集中的分布比例保持一致。

需要说明的是，由于验证集样本数量较少，单次评估结果可能受到少量样本

波动影响，因此本文更关注在相同数据划分条件下，不同模型压缩方法之间的相对性能差异。

2.1.5 数据集特点与挑战

由于 NEU-DET 原始图像分辨率仅为 200×200 像素，部分缺陷目标本身面积较小，纹理特征较弱，边界也不够清晰。因此，模型训练时需要将图像统一缩放到指定输入尺寸。本项目分别采用 416×416 和 640×640 两种输入分辨率进行实验，以分析不同输入尺寸对检测精度、计算量和推理速度的影响。较高分辨率能够保留更多细节信息，有利于模型识别弱纹理和小目标缺陷，但也会增加计算开销；较低分辨率则更有利于提升推理速度，更适合实时性要求较高的边缘部署场景。

综上，NEU-DET 数据集具有类别均衡、标注规范和工业场景真实等优点，同时也存在图像分辨率低、缺陷纹理弱、部分类别边界模糊等挑战。该数据集能够较好地检验轻量化目标检测模型在工业质检场景中的精度保持能力、模型压缩效果和实时部署潜力。

2.2 基线检测模型：YOLOv8n

2.2.1 模型选择

本实验选择 Ultralytics YOLOv8n 作为钢材表面缺陷检测任务的基线模型。YOLO 系列模型是一类典型的单阶段目标检测模型，能够在一次前向传播过程中同时完成目标类别预测和边界框回归，因此具有较高的推理效率，适合实时目标检测任务。与两阶段检测方法相比，YOLO 模型在速度方面更具优势，能够较好地满足工业现场对实时检测的需求。

在 YOLOv8 系列模型中，YOLOv8n 是参数量较小、计算量较低的轻量化版本。对于本项目所关注的边缘部署和实时检测场景而言，直接使用过大的检测模型虽然可能获得更高精度，但会带来更高的计算开销和更慢的推理速度，不利于后续在 CPU 或工业边缘设备上部署。因此，本实验选取 YOLOv8n 作为基线模型，并在此基础上进一步开展知识蒸馏和结构化剪枝实验，以评估不同模型压缩方法对检测精度、计算复杂度和实时性的影响。

2.2.2 模型结构概述

YOLOv8n 主要由 Backbone、Neck 和 Detection Head 三部分组成。其中, Backbone 用于从输入图像中提取多层次视觉特征; Neck 用于融合不同尺度的特征信息, 使模型能够同时关注大目标和小目标; Detection Head 则负责输出目标类别、置信度和边界框位置。由于 NEU-DET 数据集中不同类别缺陷在尺寸、形态和纹理强度上存在差异, 多尺度特征融合结构有助于模型同时识别大面积斑块、细长划痕以及较小的麻面坑蚀等缺陷目标。

YOLOv8n 相比更大的 YOLOv8 系列模型具有更少的参数量和更低的计算量。在本实验中, YOLOv8n 基线模型在 640×640 输入尺寸下共有 73 层, 参数量约为 3.01M, 计算量约为 8.1 GFLOPs。该模型在保持较低复杂度的同时, 仍然具备较完整的目标检测能力, 因此适合作为后续轻量化压缩实验的基准模型。

2.2.3 训练设置

本实验基于 NEU-DET 钢材表面缺陷数据集对 YOLOv8n 进行迁移学习训练。训练过程中加载 YOLOv8n 预训练权重进行初始化, 以利用其在通用视觉任务中学习到的基础特征表示, 加快模型收敛速度, 并提升小规模工业缺陷数据集上的检测效果。

为分析输入分辨率对模型检测精度和推理效率的影响, 本实验分别设置 416×416 和 640×640 两种输入尺寸进行训练与评估。其中, 416×416 输入尺寸具有较低计算量和较快推理速度, 更适合实时性要求较高的部署场景; 640×640 输入尺寸能够保留更多图像细节信息, 有利于模型识别纹理较弱、边界较模糊的缺陷目标, 但会增加计算开销。

基线模型训练轮数设置为 50 epochs, batch size 设置为 16。训练环境主要包括 Python 3.8.20、PyTorch 2.4.1+cpu、Ultralytics 8.4.56。实验主要在 CPU 环境下完成, 处理器为 AMD Ryzen 7 8845H w/ Radeon 780M Graphics。由于训练设备算力有限, 较高输入分辨率下训练耗时较长, 因此本文不将训练耗时作为核心评价指标, 而主要关注模型在验证集上的检测精度、参数量、计算量和推理速度。

2.2.4 数据增强策略

为了提高模型对不同缺陷形态、尺度变化和背景纹理差异的适应能力, 训练

过程中采用了多种数据增强策略，包括 Mosaic 增强、随机缩放、水平翻转以及 RandAugment 等方法。其中，Mosaic 增强能够将多张训练图像拼接到同一输入样本中，使模型在训练阶段接触到更加丰富的目标组合和背景变化，有助于提升模型对小目标、多目标和复杂纹理背景的鲁棒性。

对于 NEU-DET 数据集而言，部分缺陷类别具有弱纹理、小尺度或边界模糊等特点，单纯依赖原始图像训练可能导致模型泛化能力不足。通过数据增强，可以在不额外采集新图像的情况下扩展训练样本的变化形式，使模型更充分地学习不同缺陷类别的视觉特征。

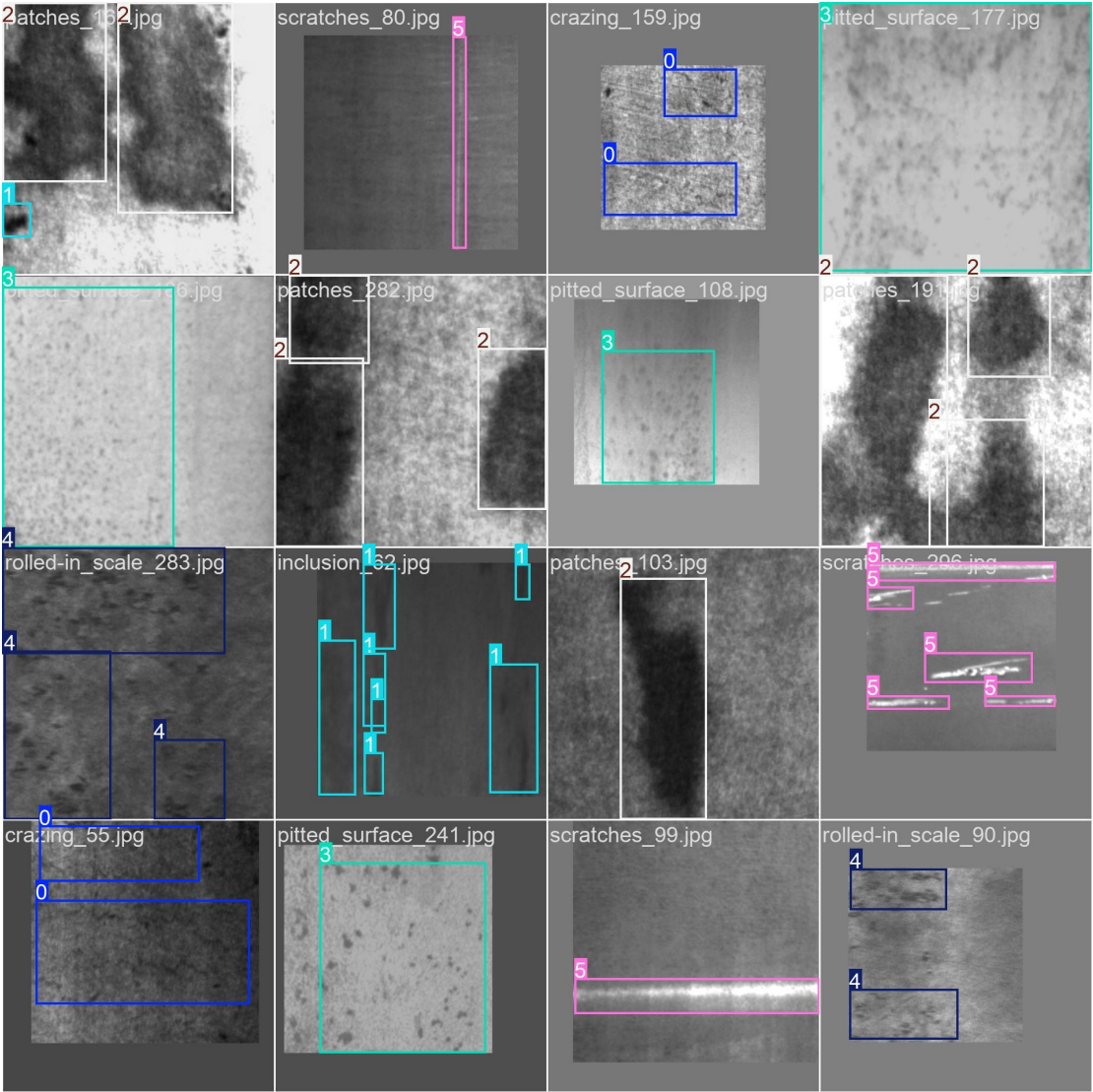


图 2 YOLOv8n 训练过程中的数据增强样例

从图 2 可以看出，经过数据增强后的训练样本中，不同类别的钢材缺陷被组

合到同一批次图像中，且目标位置、尺度和背景区域均发生一定变化。这种训练方式能够增加样本多样性，降低模型对固定图像分布的依赖，从而提升模型在验证集上的检测稳定性。

2.2.5 模型导出与部署准备

为了便于后续部署和推理加速实验，本实验将训练得到的最佳 PyTorch 权重文件 `best.pt` 导出为 ONNX 格式模型 `best.onnx`。ONNX 是一种常用的跨平台深度学习模型表示格式，能够支持后续在 ONNX Runtime、OpenVINO 或其他推理框架中进行部署测试。

本次导出的 ONNX 模型主要用于模型格式转换和推理端适配，并不等同于量化或剪枝后的压缩模型。也就是说，ONNX 导出本身不会显著改变模型参数数量和检测精度，而是为后续进一步进行 INT8 量化、边缘端部署或推理速度测试提供基础。由于本项目的主要任务是比较基线模型、知识蒸馏模型和剪枝模型之间的性能差异，因此 ONNX 导出部分作为部署准备工作进行说明，具体性能对比仍以 PyTorch 模型评估结果为主。

2.3 知识蒸馏方法

2.3.1 方法原理

知识蒸馏是一种常用的模型压缩与性能迁移方法，其核心思想是利用性能较强、参数量较大的教师模型指导参数量较小的学生模型训练，使学生模型在保持轻量化结构的同时尽可能学习教师模型的预测能力。与直接训练学生模型相比，知识蒸馏不仅使用数据集中的真实标签作为监督信号，还利用教师模型输出的软标签信息，使学生模型能够学习到不同类别之间更细致的概率分布关系。

在目标检测任务中，教师模型通常具有更强的特征提取能力和更稳定的预测输出。对于钢材表面缺陷检测任务而言，部分缺陷类别存在纹理较弱、边界模糊、小目标占比高等问题，仅依靠真实标签进行训练可能难以充分学习缺陷之间的细微差异。通过知识蒸馏，学生模型可以在训练过程中参考教师模型对目标区域、类别分布和置信度的判断，从而提升轻量模型的检测能力。

本实验采用“教师模型—学生模型”的蒸馏框架。其中，教师模型负责提供

更强的监督信息，学生模型负责学习教师模型输出并最终用于轻量化部署。蒸馏完成后，实际推理阶段只使用学生模型，不需要加载教师模型，因此不会显著增加部署阶段的参数量和计算开销。

2.3.2 教师模型与学生模型设置

本实验采用 YOLOv8m 作为教师模型，YOLOv8n 作为学生模型。YOLOv8m 相比 YOLOv8n 具有更大的参数规模和更强的特征表达能力，能够为学生模型提供更丰富的检测知识；YOLOv8n 则具有更小的参数量和更低的计算复杂度，更适合作为最终部署模型。

在蒸馏实验中，分别针对 416×416 和 640×640 两种输入分辨率进行训练与验证。教师模型训练阶段使用 YOLOv8m 预训练权重进行初始化，并在 NEU-DET 数据集上进行训练，以获得对应分辨率下的教师模型。学生模型蒸馏阶段则以对应分辨率下训练好的 YOLOv8n baseline 作为初始化模型，并在教师模型指导下继续训练。

蒸馏实验的基本设置如表 2 所示。

表 2 知识蒸馏实验基本设置

项目	设置
教师模型	YOLOv8m
学生模型	YOLOv8n
数据集	NEU-DET 钢材表面缺陷数据集
输入尺寸	416×416 、 640×640
教师模型训练轮数	20 epochs
学生模型初始化	对应分辨率下的 YOLOv8n baseline 权重
教师模型参数状态	冻结参数，不参与反向更新
学生模型参数状态	参与训练并更新参数
蒸馏损失类型	检测损失 + KL 散度蒸馏损失
温度系数	6.0
蒸馏损失权重	0.7

2.3.3 蒸馏损失设计

在普通目标检测训练中，模型主要通过真实标签计算检测损失，包括目标边界框定位损失、类别预测损失和置信度相关损失。知识蒸馏在此基础上进一步引入教师模型输出作为额外监督信号，使学生模型不仅学习真实标注信息，也学习教师模型的预测分布。

本实验中的蒸馏损失主要由两部分组成：第一部分为 YOLOv8 原始检测损失，用于保证学生模型能够完成基本的缺陷定位与分类任务；第二部分为基于 KL 散度的蒸馏损失，用于约束学生模型输出分布接近教师模型输出分布。整体损失可以表示为：

$$L_{\text{total}} = L_{\text{det}} + \alpha L_{\text{KD}}$$

其中， L_{total} 表示学生模型训练过程中的总损失， L_{det} 表示原始目标检测损失， L_{KD} 表示蒸馏损失， α 表示蒸馏损失权重。本实验中 α 设置为 0.7，用于控制教师模型监督信号在总训练目标中的影响程度。

在计算蒸馏损失时，引入温度系数 T 对教师模型和学生模型的输出分布进行平滑处理。本实验中温度系数设置为 6.0。较高的温度系数能够使类别概率分布更加平滑，使学生模型更容易学习教师模型在不同类别之间的相对判断关系，而不仅仅是学习最终预测类别。

2.3.4 蒸馏训练流程

知识蒸馏训练过程主要包括以下步骤。

首先，在 NEU-DET 数据集上分别训练 416×416 和 640×640 输入尺寸下的 YOLOv8m 教师模型。教师模型训练完成后，保留其最佳权重文件，用于后续指导学生模型训练。

其次，加载对应输入尺寸下的 YOLOv8n baseline 作为学生模型初始权重。这样做的目的在于使学生模型已经具备基本的钢材表面缺陷检测能力，再通过教师模型进一步优化其预测分布。

然后，在学生模型蒸馏训练阶段冻结教师模型参数，仅更新学生模型参数。每次训练迭代中，教师模型和学生模型同时对输入图像进行前向传播，教师模型输出作为软监督信息，学生模型则同时受到真实标签和教师输出的约束。通过检测损失和蒸馏损失的联合优化，学生模型逐步学习教师模型的检测知识。

最后，蒸馏完成后仅保留学生模型用于验证与部署。由于学生模型仍然是

YOLOv8n 结构，因此蒸馏方法不会改变其主体网络规模。也就是说，知识蒸馏的主要作用不是直接减少参数量，而是在保持轻量模型结构的前提下改善或稳定学生模型的检测性能。

2.3.5 方法特点与适用性分析

知识蒸馏的优势在于能够在不显著增加学生模型推理复杂度的情况下提升轻量模型性能。对于本项目而言，YOLOv8n 本身已经是较轻量的检测模型，直接进一步减小模型结构可能导致特征提取能力不足，而知识蒸馏可以通过教师模型提供额外监督，使学生模型在保持较小参数量的同时获得更好的检测表现。

但是，知识蒸馏的效果并不是必然提升的，它与教师模型自身性能密切相关。如果教师模型在某一输入分辨率下的检测效果不如学生模型 baseline，则教师模型提供的监督信息可能不足以改善学生模型表现，甚至可能对学生模型产生负面影响。因此，在后续实验分析中，需要分别比较 416×416 和 640×640 两种分辨率下教师模型、学生 baseline 和蒸馏后学生模型的检测结果，以判断知识蒸馏是否真正带来了性能增益。

综合来看，知识蒸馏适合作为轻量级目标检测模型的性能增强方法。它与结构化剪枝的优化方向不同：蒸馏侧重于提升或保持学生模型的检测精度，剪枝侧重于减少模型参数量和计算量。因此，二者可以从不同角度服务于轻量级实时目标检测优化任务。

2.4 结构化通道剪枝方法

2.4.1 方法原理

结构化剪枝是一种常用的模型压缩方法，其核心思想是根据一定的重要性评价标准，移除神经网络中贡献较小的结构单元，从而降低模型参数量和计算量。与非结构化剪枝只将部分权重置零不同，结构化剪枝通常直接删除卷积通道、卷积核或网络层等完整结构，因此剪枝后的模型在实际推理框架中更容易获得速度提升，也更适合部署到边缘设备或工业现场终端。

对于卷积神经网络而言，不同通道对最终检测结果的贡献并不完全相同。一些通道能够提取关键纹理、边缘和目标形态信息，而另一些通道可能存在冗余。

结构化通道剪枝通过评估通道重要性，删除相对不重要的通道，使网络结构更加紧凑。剪枝后模型的参数量和计算量会减少，但如果剪枝比例过高，也可能破坏模型的特征表达能力，导致检测精度下降。因此，结构化剪枝需要在压缩率和检测精度之间进行权衡。

在本实验中，结构化剪枝主要用于压缩 YOLOv8n 模型，以减少模型参数量和计算量，并分析不同剪枝比例对钢材表面缺陷检测效果的影响。由于 NEU-DET 数据集中存在弱纹理、小目标和边界模糊等问题，模型剪枝后可能更容易丢失细节特征，因此剪枝后需进一步进行微调训练，以恢复模型的检测性能。

2.4.2 剪枝对象与基本设置

本实验以 YOLOv8n 作为剪枝对象，分别在 416×416 和 640×640 两种输入分辨率下构建结构化剪枝实验，并设置 0.3 和 0.5 两种剪枝比例进行对比。其中， 416×416 输入分辨率下的剪枝实验主要用于分析较低计算量场景中的模型压缩效果， 640×640 输入分辨率下的剪枝实验则用于分析较高输入分辨率下模型压缩后精度保持与计算量变化情况。剪枝比例 0.3 表示相对保守的通道压缩，目标是在降低模型复杂度的同时尽量保持检测精度；剪枝比例 0.5 表示更强的压缩设置，目标是进一步减少参数量和计算量，但精度下降风险也相应增加。

剪枝过程中使用 torch-pruning 工具对 YOLOv8n 进行结构化通道剪枝。为了避免破坏目标检测输出结构，剪枝时保留检测头相关结构，不直接对 Detect 层进行剪枝。这样可以在减少主干网络和特征融合部分冗余通道的同时，保证模型最终输出维度仍然与目标检测任务相匹配。剪枝实验的基本设置如表 3 所示。

表 3 结构化剪枝实验基本设置

项目	设置
剪枝对象	YOLOv8n baseline 模型
数据集	NEU-DET 钢材表面缺陷数据集
输入尺寸	416×416 、 640×640
剪枝方法	结构化通道剪枝
剪枝工具	torch-pruning
通道重要性评价	Group Magnitude Importance
剪枝比例	0.3、0.5

忽略层	Detect 检测层
剪枝后处理	对应分辨率下微调训练恢复精度
评价指标	mAP、Params、FLOPs、FPS

2.4.3 YOLOv8n 模块结构与剪枝适配

YOLOv8n 网络结构中包含 C2f 模块，该模块是 YOLOv8 系列模型中的重要特征提取结构，主要用于增强网络中的特征传递与梯度流动能力。由于结构化通道剪枝需要根据模型计算图分析不同卷积层和通道之间的依赖关系，因此在对 YOLOv8n 进行剪枝时，需要注意网络内部模块之间的通道连接关系，避免剪枝后出现结构不匹配或模型无法正常推理的问题。

在本实验的剪枝实现过程中，针对 YOLOv8n 中部分模块的通道依赖关系进行了适配处理。具体而言，剪枝脚本在执行结构化剪枝前，对 C2f 模块进行了适配性处理，使其更便于 torch-pruning 构建通道依赖关系并执行通道裁剪。该步骤的目的并不是直接提升模型检测精度，而是保证剪枝过程能够顺利执行，并尽量避免剪枝后因通道维度不一致导致模型加载或推理错误。

完成模块适配后，模型仍然保持 YOLOv8n 的整体检测框架，后续剪枝主要作用于主干网络和特征融合部分的冗余通道，同时保留 Detect 检测层结构，以保证最终输出仍然符合目标检测任务要求。

2.4.4 剪枝流程

结构化剪枝流程主要包括以下几个步骤。

首先，分别加载 416×416 和 640×640 输入分辨率下训练好的 YOLOv8n baseline 权重，并将模型中的 C2f 模块替换为 C2f_v2 模块。替换完成后，将模型设置为可剪枝状态，并针对对应输入尺寸构造示例输入，用于建立模型计算图和通道依赖关系。

其次，设置剪枝过程中的忽略层。本实验将 Detect 检测层加入忽略列表，避免剪枝直接改变检测输出结构。由于检测头与类别数、边界框输出维度密切相关，如果对其进行不当剪枝，可能会导致模型输出维度错误，因此需要在剪枝过程中进行保护。

然后，使用通道重要性评价方法计算不同通道的重要程度，并根据设定的剪枝比例删除相对不重要的通道。本实验分别设置 0.3 和 0.5 两种剪枝比例，观

察不同压缩强度对参数量、计算量和检测性能的影响。剪枝完成后，统计剪枝前后的参数量和计算量，以验证剪枝是否真正生效。

最后，保存剪枝后的模型权重，并对剪枝模型进行微调训练。由于剪枝会直接改变模型结构，部分原有特征通道被移除，模型性能可能明显下降。通过微调训练，剪枝后的模型能够在新的网络结构下重新适应 NEU-DET 数据集，从而恢复一定的检测精度。

2.4.5 剪枝后微调

剪枝后微调是结构化剪枝实验中非常关键的一步。直接剪枝得到的模型虽然参数量和计算量下降，但由于部分通道被移除，模型原有的特征提取能力会受到影响。如果不进行微调，模型可能出现检测精度明显下降，甚至无法有效识别缺陷目标的情况。

本实验在完成结构化剪枝后，对剪枝模型进行继续训练，使模型在保留压缩结构的基础上重新学习钢材表面缺陷特征。微调过程中，模型参数会根据 NEU-DET 训练集继续更新，以适应剪枝后的网络结构。微调完成后，再在验证集上统一评估模型的 Precision、Recall、mAP、参数量、FLOPs 和 FPS 等指标。

通过剪枝后微调，可以在一定程度上恢复模型的检测能力，使剪枝模型不仅具有较低的复杂度，也具备可用的检测性能。因此，在后续实验结果分析中，本文主要比较微调后的剪枝模型与基线模型、蒸馏模型之间的性能差异，而不将未经微调的原始剪枝模型作为最终部署结果。

2.4.6 方法特点与适用性分析

结构化通道剪枝的主要优势在于能够直接减少模型参数量和计算量，使模型结构更加紧凑。与知识蒸馏不同，剪枝方法能够改变模型本身的结构规模，因此更适合用于降低模型存储占用和计算开销。在边缘设备或工业现场终端中，参数量和 FLOPs 的降低有助于提升模型部署可行性。

但是，结构化剪枝也存在一定风险。对于钢材表面缺陷检测任务而言，部分类别缺陷本身具有弱纹理、小尺度和边界模糊等特点。如果剪枝比例过高，模型可能会删除对细节特征有贡献的通道，从而影响对这些困难类别的检测能力。因此，剪枝比例需要谨慎设置，并结合微调训练和实际验证结果进行综合判断。

在本项目中，分别在 416×416 和 640×640 两种输入分辨率下设置 0.3

和 0.5 两种剪枝比例，用于比较不同分辨率和不同压缩强度下模型精度、参数量、FLOPs 和 FPS 的变化，分析压缩强度对模型性能的影响，并为后续边缘部署场景选择合适的模型提供依据。整体来看，结构化剪枝适合用于减少 YOLOv8n 的模型复杂度，而剪枝后微调则是保证模型保持可用检测精度的重要步骤。

2.5 评价指标

为全面评估不同模型在钢材表面缺陷检测任务中的表现，本文从检测精度、模型复杂度和推理速度三个方面设置评价指标。检测精度方面，主要采用 Precision、Recall、mAP@0.5 和 mAP@0.5:0.95；模型复杂度方面，主要采用参数量 Params 和计算量 FLOPs；推理速度方面，主要采用 FPS 作为评价指标。

Precision 表示模型预测结果的准确程度，即模型预测为缺陷目标的结果中真正正确的比例。Precision 越高，说明模型误检越少。Recall 表示模型对真实缺陷目标的检出能力，即所有真实缺陷中被模型成功检测出的比例。Recall 越高，说明模型漏检越少。对于工业质检任务而言，漏检可能导致缺陷产品进入后续环节，因此 Recall 是评价模型实用性的重要指标。

mAP 是目标检测任务中常用的综合评价指标。本文主要采用 mAP@0.5 和 mAP@0.5:0.95 两种形式。其中，mAP@0.5 表示在 IoU 阈值为 0.5 时计算得到的平均检测精度，能够反映模型对缺陷目标的基本检测能力；mAP@0.5:0.95 则综合多个 IoU 阈值下的检测结果，对边界框定位精度要求更高，因此能够更严格地评价模型检测性能。

Params 表示模型参数量，用于衡量模型存储规模。参数量越小，模型权重文件占用空间越低，更有利于部署到存储资源有限的设备。FLOPs 表示模型完成一次前向推理所需的计算量，用于衡量模型理论计算复杂度。FLOPs 越低，说明模型计算开销越小，通常更有利于边缘端部署。

FPS 表示模型每秒能够处理的图像数量，是衡量实时检测能力的重要指标。FPS 越高，说明模型推理速度越快。需要注意的是，Params 和 FLOPs 反映的是模型结构层面的复杂度，而实际 FPS 还会受到硬件平台、推理框架、输入尺寸和算子优化等因素影响。因此，本文在实验分析中同时比较 mAP、Params、FLOPs 和 FPS，以综合判断不同模型在精度、复杂度和实时性之间的平衡关系。

3 实验与结果分析

3.1 实验环境与统一设置

为保证不同模型实验结果之间具有可比性, 本文在相同数据集划分和相同评价指标体系下, 对 YOLOv8n 基线模型、知识蒸馏模型和结构化剪枝模型进行统一评估。实验数据集均采用 NEU-DET 钢材表面缺陷数据集, 训练集和验证集划分方式与 2.1 节保持一致。模型评估主要围绕检测精度、模型复杂度和推理速度展开, 重点比较不同输入分辨率和不同压缩方法下模型性能的变化。

本实验主要使用 Python、PyTorch 和 Ultralytics YOLO 框架完成模型训练、验证、蒸馏、剪枝和推理测试。由于实验主要在 CPU 环境下完成, 训练和推理速度会受到硬件算力限制。因此, 本文更关注不同模型在相同环境下的相对性能差异, 而不是将训练耗时作为主要评价指标。

表 4 实验环境与基本设置

项目	设置
操作系统	Windows
编程语言	Python 3.8.20
深度学习框架	PyTorch 2.4.1+cpu
目标检测框架	Ultralytics 8.4.56
主要模型	YOLOv8n、YOLOv8m
数据集	NEU-DET 钢材表面缺陷数据集
运行设备	CPU
处理器	AMD Ryzen 7 8845H w/ Radeon 780M Graphics
输入尺寸	416×416 、 640×640
训练轮数	50 epochs
Batch size	16
主要压缩方法	知识蒸馏、结构化通道剪枝
主要评价指标	Precision、Recall、mAP、Params、FLOPs、FPS

在实验设计上, 本文首先训练 YOLOv8n 基线模型, 分别获得 416×416 和 640×640 两种输入分辨率下的 baseline 结果。随后, 在相同数据集划分条件下

进行知识蒸馏实验，使用 YOLOv8m 作为教师模型、YOLOv8n 作为学生模型，比较蒸馏前后学生模型检测性能的变化。最后，对 YOLOv8n 进行结构化通道剪枝，并设置 0.3 和 0.5 两种剪枝比例，比较不同剪枝强度下模型参数量、计算量、检测精度和推理速度的变化。

为减少实验设置差异对结果造成的影响，后续所有模型均在同一验证集上进行评估。验证集共包含 30 张图像和 64 个缺陷目标实例，涵盖 crazing、inclusion、patches、pitted_surface、rolled-in_scale 和 scratches 六类缺陷。由于验证集样本数量较少，单个类别的指标可能存在一定波动，因此本文重点关注不同模型之间的整体指标变化趋势，以及在相同验证条件下压缩前后模型的相对性能差异。

3.2 Baseline 实验结果与分辨率影响分析

为评估 YOLOv8n 在 NEU-DET 钢材表面缺陷检测任务中的基础性能，本文分别在 416×416 和 640×640 两种输入分辨率下进行 baseline 实验。两组实验均采用相同的数据集划分方式和评价指标，训练轮数均为 50 epochs。通过对比两种输入尺寸下的 Precision、Recall、mAP、推理耗时、FPS 和训练时间，可以分析输入分辨率对模型检测精度和运行效率的影响。

416×416 输入尺寸具有较低的计算开销和更快的推理速度，更适合实时工业检测和边缘端部署； 640×640 输入尺寸能够保留更多图像细节，理论上有利于提升缺陷目标的检测效果。因此，本节重点分析两种输入分辨率在精度和速度之间的权衡关系。

3.2.1 不同输入分辨率下的整体性能对比

表 5 汇总了 YOLOv8n 在 416×416 和 640×640 两种输入分辨率下的 baseline 实验结果。

表 5 YOLOv8n 基线模型在不同输入分辨率下的性能对比

指标	416×416 YOLOv8n	640×640 YOLOv8n	对比说明
Epochs	50	50	训练轮数相同
Precision	0.726	0.726	基本相同
Recall	0.784	0.790	640×640 提高 0.006
mAP@0.5	0.827	0.835	640×640 提高 0.008
mAP@0.5:0.95	0.502	0.497	416×416 提高 0.005

Inference time / image	15.3 ms	51.2 ms	416 × 416 推理更快
Total time / image	≈ 16.1 ms	≈ 52.7 ms	416 × 416 总耗时更低
Estimated FPS	36.65	20.34	416 × 416 约为 640 × 640 的 3.3 倍
Parameters	3,006,818	3,006,818	模型结构相同
Training time	1.306 h	> 3 h, 未完整记录	640 × 640 训练更慢
结果特征	推理速度较快	检测精度略高	二者分别体现不同输入分辨率下的效率与精度特征

注：表 5 中的速度来自 YOLO 验证日志，用于 baseline 分辨率对比。

从表 5 可以看出，两种输入分辨率下 YOLOv8n 的参数量相同，说明模型结构保持一致，性能差异主要来自输入图像尺寸的变化，而不是模型规模变化。因此，该实验能够较直接地反映输入分辨率对检测精度和推理效率的影响。

3.2.2 输入分辨率对检测精度的影响

从检测精度来看，640 × 640 输入尺寸在 Recall 和 mAP@0.5 上略优于 416 × 416。具体而言，Recall 由 0.784 提升至 0.790，提升 0.006；mAP@0.5 由 0.827 提升至 0.835，提升 0.008。这说明更高输入分辨率能够在一定程度上保留更多图像细节，对缺陷目标的检出能力和基本定位分类效果有轻微提升。

但是，这种精度提升幅度较小，并不构成明显优势。同时，640 × 640 在 mAP@0.5:0.95 上并未优于 416 × 416。416 × 416 的 mAP@0.5:0.95 为 0.502，而 640 × 640 为 0.497，416 × 416 反而略高 0.005。mAP@0.5:0.95 使用多个 IoU 阈值综合评价模型定位质量，因此该结果说明提高输入分辨率并没有在更严格的定位评价标准下带来稳定提升。

这表明，在本实验中，单纯提高输入分辨率带来的精度收益有限。对于 NEU-DET 钢材表面缺陷检测任务而言，640 × 640 虽然能够略微提升 Recall 和 mAP@0.5，但并没有显著改善模型整体检测能力。

3.2.3 输入分辨率对推理效率的影响

与检测精度相比，输入分辨率对推理效率的影响更加明显。416 × 416 输入尺寸下，模型单张图像推理耗时为 15.3 ms，总耗时约为 16.1 ms，Estimated FPS 约为 62.1；而 640 × 640 输入尺寸下，模型单张图像推理耗时为 51.2 ms，总耗时约为 52.7 ms，Estimated FPS 约为 19.0。

也就是说， 416×416 的推理速度约为 640×640 的 1.8 倍。在检测精度差距较小的情况下， 416×416 能够显著降低推理延迟，提高检测吞吐率，更符合实时工业质检场景对速度的要求。

此外， 416×416 的训练时间约为 1.306 h，而 640×640 训练时间超过 3 h，说明较高输入分辨率不仅会增加推理成本，也会明显增加训练成本。因此，从训练效率和部署效率角度看， 416×416 在实时检测和边缘部署场景中具有更明显的效率优势。

为更直观展示两种输入分辨率在检测精度和推理速度之间的差异，本文绘制 mAP@0.5 与 FPS 对比图，如图 3 所示。

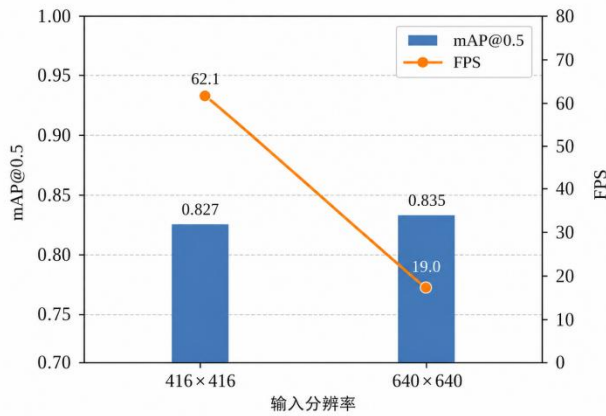


图 3 YOLOv8n 基线模型在不同输入分辨率下的 mAP@0.5 与 FPS 对比

图 3 进一步直观展示了两种输入分辨率下精度与速度之间的差异。 640×640 的 mAP@0.5 略高于 416×416 ，但 FPS 明显低于 416×416 ，说明提高输入分辨率带来的精度收益有限，而推理效率下降较为明显。

3.2.4 本节小结

综合以上实验结果， 416×416 和 640×640 下 YOLOv8n 均能够完成 NEU-DET 钢材表面缺陷检测任务，但二者在精度和速度上的表现存在明显差异。

640×640 在 Recall 和 mAP@0.5 上略高，说明较高输入分辨率对缺陷检出有一定帮助，但提升幅度较小；同时，其 mAP@0.5:0.95 略低于 416×416 ，说明高分辨率并未在严格定位评价指标上形成稳定优势。相比之下， 416×416 在保持接近检测精度的同时，推理耗时更低，FPS 达到约 62.1，在实时检测和边缘部署场景中具有明显效率优势。

因此，本文将 416×416 和 640×640 两种输入分辨率均作为 baseline 实验的重要组成部分。 416×416 主要体现低输入分辨率下模型的推理效率和实时检测能力， 640×640 实验则体现较高输入分辨率下模型对图像细节和检测精度的影

响。二者而是从不同角度共同构成 YOLOv8n 基线性能分析的依据。通过对两种分辨率下的综合比较,可以更全面地评估输入分辨率变化对钢材表面缺陷检测任务的影响,并为后续模型优化实验提供统一、可靠的 baseline 参考。

3.3 训练收敛性分析

为进一步验证 YOLOv8n baseline 模型训练过程的稳定性,本文对训练过程中损失函数和检测指标的变化趋势进行分析。图 4 和图 5 分别展示了两组 baseline 实验在 50 个 epoch 内的训练损失、验证损失以及检测指标变化情况。

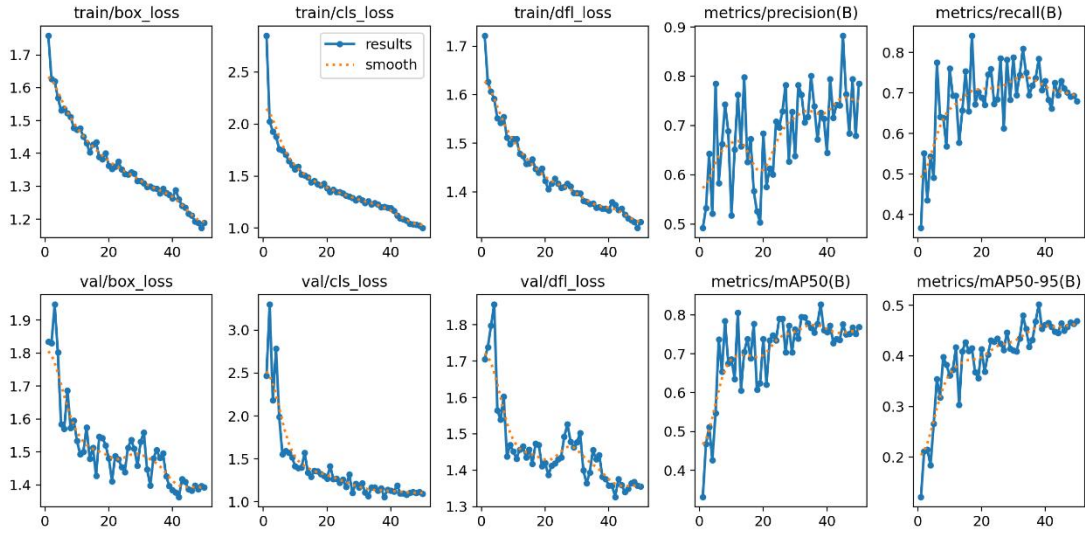


图 4 YOLOv8n-416 基线模型训练过程中的损失与检测指标变化曲线

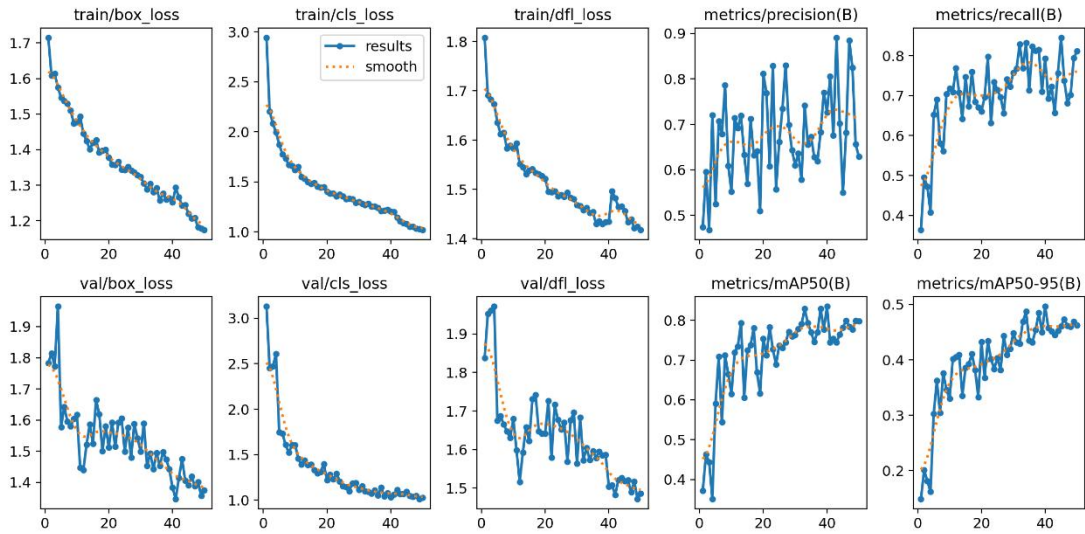


图 5 YOLOv8n-640 基线模型训练过程中的损失与检测指标变化曲线

从训练损失来看,两组 baseline 实验中的 train/box_loss、train/cls_loss 和

train/df_l_loss 均随 epoch 增加呈明显下降趋势。其中, box_loss 的下降说明模型对缺陷目标边界框位置的回归能力逐步增强; cls_loss 的下降说明模型对不同缺陷类别的分类能力不断提高; df_l_loss 的下降则表明模型对边界框分布的学习逐渐稳定。整体来看, 训练集损失曲线下下降较为连续, 说明模型在训练过程中能够有效学习钢材表面缺陷特征。

从验证损失来看, val/box_loss、val/cls_loss 和 val/df_l_loss 整体也呈下降趋势, 说明模型在验证集上同样具有较好的收敛表现, 并未出现明显的训练集损失下降而验证集损失持续上升的情况。因此, 从损失变化趋势判断, baseline 模型训练过程整体稳定, 未表现出明显的持续性过拟合趋势。

从检测指标来看, precision 和 recall 在波动中整体提升, mAP@0.5 和 mAP@0.5:0.95 呈现较为明显的上升趋势。其中, mAP@0.5 和 mAP@0.5:0.95 在训练前期提升较快, 说明模型能够较快学习到主要缺陷特征; 随着训练轮数增加, 指标逐渐趋于稳定, 表明模型检测性能逐步收敛。

需要注意的是, precision、recall 以及部分验证损失曲线在训练后期存在一定波动。这种波动可能与验证集样本数量有限、缺陷类别分布不均衡以及部分缺陷纹理相似度较高有关。由于整体趋势仍表现为损失下降、检测指标提升, 因此该波动属于可接受范围, 并不影响 baseline 模型整体收敛性的判断。

综合来看, 两组 baseline 实验的训练损失与验证损失整体下降, mAP 指标逐步提升并趋于稳定, 说明 YOLOv8n 在 NEU-DET 钢材表面缺陷检测任务上能够实现有效收敛。该结果表明, 两组 baseline 模型均具有较稳定的训练过程, 可作为后续模型优化实验的可靠基础。

3.4 知识蒸馏实验结果

为分析知识蒸馏方法对 YOLOv8n 学生模型检测性能的影响, 本文分别在 416×416 和 640×640 两种输入分辨率下进行蒸馏实验, 并将基线模型、教师模型和蒸馏后学生模型进行对比。知识蒸馏的目的不是改变学生模型结构, 而是通过教师模型提供的监督信息改善学生模型的特征学习能力。因此, 本节重点从 mAP@0.5、参数量和推理效率三个方面分析蒸馏前后模型性能变化。

3.4.1 蒸馏实验整体性能对比

表 6 汇总了不同输入分辨率下基线模型、教师模型和蒸馏后学生模型的主要实验结果。由于蒸馏实验中不同模型的 Precision、Recall 和 mAP@0.5:0.95 信息并不完全一致，因此表 6 主要选取各组实验均可比较的 mAP@0.5、参数量和推理耗时作为核心指标。

表 6 知识蒸馏实验性能对比

输入分辨率	模型	mAP@0.5	参数量	推理耗时 / image	Estimated FPS
416 × 416	基线模型 YOLOv8n	0.827	3.0M	28.7 ms	≈34.8
416 × 416	教师模型 YOLOv8m	0.810	25.8M	226.4 ms	≈4.4
416 × 416	蒸馏后 YOLOv8n	0.812	3.0M	38.8 ms	≈27
640 × 640	基线模型 YOLOv8n	0.835	3.0M	73.7 ms	≈13.6
640 × 640	教师模型 YOLOv8m	0.844	25.8M	443.0 ms	≈2.3
640 × 640	蒸馏后 YOLOv8n	0.808	3.0M	79.3 ms	≈21

注：表中推理耗时采用蒸馏实验同一组验证结果中的记录，仅用于比较本节不同模型之间的推理速度差异；Estimated FPS 根据单张图像推理耗时估算得到。

从表 6 可以看出，蒸馏后模型仍然采用 YOLOv8n 作为学生模型，其参数量为 3.0M，与基线模型保持一致，说明知识蒸馏并不会改变学生模型的网络结构，也不会增加学生模型参数量。相比之下，教师模型 YOLOv8m 的参数量为 25.8M，明显大于学生模型，推理耗时也显著增加。因此，教师模型不适合作为轻量化部署模型，其主要作用是为学生模型训练提供蒸馏监督。

3.4.2 蒸馏对检测精度的影响

从 mAP@0.5 来看，知识蒸馏在两种输入分辨率下表现并不完全一致。在 416 × 416 输入分辨率下，基线模型 mAP@0.5 为 0.827，教师模型为 0.810，蒸馏后学生模型为 0.812，精度较基线略有下降。蒸馏后模型的 mAP@0.5 低于基

线模型, 说明该组蒸馏实验并未提升 416×416 下 YOLOv8n 的整体检测精度。

结合 416×416 下 Precision 和 Recall 的变化可以进一步看出, 蒸馏后 YOLOv8n 的 Precision 由 0.726 提高至 0.906, 说明模型预测结果的准确性提高, 误检现象有所减少; 但 Recall 由 0.784 下降至 0.661, 说明模型对缺陷目标的检出能力下降, 漏检情况增加。由于 Recall 下降幅度较大, 最终导致 mAP@0.5 从 0.827 降至 0.784。

在 640×640 输入分辨率下, 基线模型 mAP@0.5 为 0.835, 教师模型为 0.844, 蒸馏后学生模型为 0.808。蒸馏后模型的 mAP@0.5 略高于基线模型, 但提升幅度仅为 0.001, 说明蒸馏在 640×640 下带来了非常有限的精度改善。从数值上看, 640×640 蒸馏模型的 mAP@0.5 高于 416×416 蒸馏模型, 并且更接近对应输入分辨率下的教师模型性能。

3.4.3 蒸馏对模型规模和推理效率的影响

从模型规模来看, 416×416 和 640×640 两种输入分辨率下, 蒸馏后学生模型的参数量均为 3.0M, 与 YOLOv8n 基线模型一致。这说明知识蒸馏本身不是模型压缩结构操作, 而是一种训练策略优化方法, 其作用主要体现在尝试改善学生模型的检测性能, 而不是减少参数量。

从推理效率来看, 蒸馏后学生模型的推理耗时高于对应的基线模型。在 416×416 下, 基线模型推理耗时为 28.7 ms, 蒸馏后模型为 38.8 ms; 在 640×640 下, 基线模型推理耗时为 73.7 ms, 蒸馏后模型为 79.3 ms。虽然蒸馏后模型结构和参数量没有变化, 但实际推理耗时仍可能受到 CPU 运行状态、缓存、数据读取、预处理和后处理过程等因素影响。

与教师模型相比, 蒸馏后学生模型仍然具有明显的轻量化优势。 416×416 下教师模型推理耗时为 226.4 ms, 而蒸馏后学生模型为 38.8 ms; 640×640 下教师模型推理耗时为 443.0 ms, 而蒸馏后学生模型为 79.3 ms。由此可见, 蒸馏后学生模型虽然不能完全达到教师模型的精度水平, 但在参数量和推理速度方面明显优于教师模型。

为进一步展示不同模型在精度和速度之间的关系, 本文绘制知识蒸馏模型的精度—速度权衡对比图, 如图 6 所示。

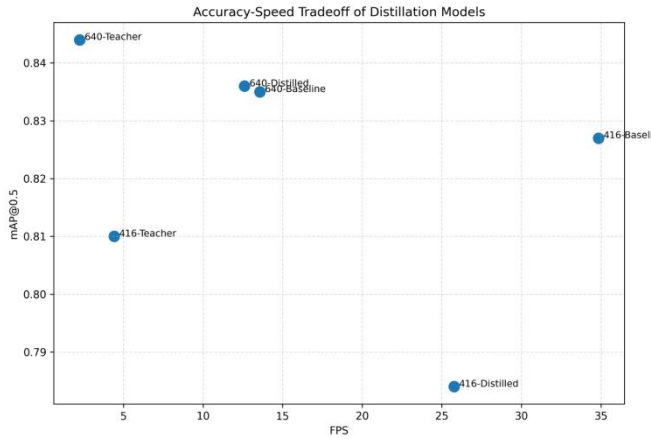


图 6 知识蒸馏模型的精度—速度权衡对比

从图 6 可以看出, YOLOv8n 学生模型整体具有更小的参数量和更快的推理速度; 教师模型虽然在部分设置下具有较高 mAP@0.5, 但推理速度明显较低。其中, 640×640 蒸馏模型的 mAP@0.5 为 0.836, 略高于 640×640 基线模型的 0.835, 并且明显快于 640×640 教师

模型, 因此在蒸馏实验中表现出较好的精度—速度平衡。

3.4.4 本节小结

综合来看, 知识蒸馏不会改变 YOLOv8n 学生模型的参数量。蒸馏后模型仍保持 3.0M 参数规模, 因此蒸馏方法的主要作用不是压缩模型, 而是在学生模型结构不变的条件下尝试改善检测精度。

从实验结果看, 416×416 蒸馏模型的 Precision 明显提升, 但 Recall 和 mAP@0.5 下降, 说明该组蒸馏结果更偏向减少误检, 但漏检增加, 整体检测精度未优于基线模型。相比之下, 640×640 蒸馏模型的 mAP@0.5 为 0.808, 与 640×640 基线模型的 0.835 略有下降, 但参数量保持 3.0M 不变。

因此, 本组实验表明, 知识蒸馏的效果与输入分辨率、教师模型性能以及蒸馏策略有关, 并不必然带来所有指标的同步提升。就当前结果而言, 640×640 蒸馏模型在保持学生模型参数量不变的前提下, mAP@0.5 与对应基线模型基本持平并略有提高, 表现出较稳定的精度—速度平衡; 而 416×416 蒸馏模型虽然 Precision 提升明显, 但 Recall 和整体 mAP 下降, 仍需要进一步优化蒸馏损失权重或训练策略。

3.5 剪枝实验结果

为分析结构化剪枝对 YOLOv8n 模型复杂度、检测精度和推理速度的影响, 本文在 baseline 模型基础上进行不同剪枝比例实验, 并比较 0.3 和 0.5 两种剪枝比例下模型的性能变化。剪枝方法的主要目的在于移除模型中的冗余通道, 降

低参数量和计算量，从而提升模型在边缘端或低算力设备上的部署效率。

与知识蒸馏不同，剪枝会直接改变模型结构和计算规模。因此，本节重点从 mAP@0.5、mAP@0.5:0.95、FLOPs、参数量和 FPS 等指标分析剪枝比例变化对模型性能的影响，并进一步讨论剪枝比例与实际检测性能之间是否存在简单线性关系。

3.5.1 剪枝实验整体性能对比

表 7 汇总了 YOLOv8n baseline、Pruned 0.3 和 Pruned 0.5 模型在 416×416 与 640×640 两种输入尺寸下的统一评估结果。

表 7 不同剪枝比例下 YOLOv8n 模型性能对比

输入尺寸	模型	mAP@0.5	mAP@0.5:0.95	FLOPs/G	参数量/M	FPS
416×416	YOLOv8n Baseline	0.827	0.502	1.709	3.007	36.65
416×416	Pruned 0.3	0.754	0.436	1.039	1.682	43.01
416×416	Pruned 0.5	0.783	0.440	0.728	1.059	50.94
640×640	YOLOv8n Baseline	0.835	0.497	4.045	3.007	20.34
640×640	Pruned 0.3	0.794	0.483	2.459	1.682	22.02
640×640	Pruned 0.5	0.804	0.451	1.722	1.059	34.96

从表 7 可以看出，0.3 和 0.5 两种剪枝比例均能够有效降低 YOLOv8n 的参数量和 FLOPs。在两种输入尺寸下，baseline 模型参数量均为约 3.007M；0.3 剪枝后参数量降低至约 1.682M，0.5 剪枝后进一步降低至约 1.059M。这说明更高剪枝比例能够带来更明显的模型压缩效果。

3.5.2 剪枝对模型压缩效果的影响

从模型规模来看，0.5 剪枝模型的参数量明显低于 0.3 剪枝模型。与 baseline 相比，0.3 剪枝使参数量由 3.007M 降低至 1.682M，参数量减少约 44.0%；0.5 剪枝进一步将参数量降低至 1.059M，参数量减少约 64.8%。因此，从参数压缩角度看，0.5 剪枝比 0.3 剪枝具有更强的压缩效果。

从 FLOPs 来看, 两种输入尺寸下也呈现相同趋势。在 416×416 输入尺寸下, baseline 的 FLOPs 为 1.709G, 0.3 剪枝后降低至 1.039G, 0.5 剪枝后进一步降低至 0.728G。在 640×640 输入尺寸下, baseline 的 FLOPs 为 4.045G, 0.3 剪枝后降低至 2.459G, 0.5 剪枝后进一步降低至 1.722G。说明结构化剪枝不仅减少了参数量, 也能够有效降低理论计算量。

因此, 对于“0.5 是否比 0.3 参数更少”这一问题, 当前实验结果给出的答案是肯定的。0.5 剪枝在两种输入尺寸下均具有更低的参数量和 FLOPs, 说明较高剪枝比例能够带来更强的模型压缩效果。

3.5.3 剪枝对检测精度和推理速度的影响

从检测精度来看, 剪枝后模型整体 mAP 相比 baseline 有所下降, 但 0.5 剪枝并不一定比 0.3 剪枝精度更差。在 416×416 输入尺寸下, baseline 的 mAP@0.5 为 0.827, 0.3 剪枝后下降至 0.754, 而 0.5 剪枝模型的 mAP@0.5 为 0.783, 高于 0.3 剪枝模型。在 640×640 输入尺寸下, baseline 的 mAP@0.5 为 0.835, 0.3 剪枝模型为 0.794, 0.5 剪枝模型为 0.804, 同样表现为 0.5 剪枝高于 0.3 剪枝。

但是, 从更严格的 mAP@0.5:0.95 指标看, 结果并不完全一致。在 416×416 输入尺寸下, 0.5 剪枝模型的 mAP@0.5:0.95 为 0.440, 略高于 0.3 剪枝模型的 0.436; 而在 640×640 输入尺寸下, 0.5 剪枝模型的 mAP@0.5:0.95 为 0.451, 低于 0.3 剪枝模型的 0.483。这说明剪枝比例对检测精度的影响并不是单一方向的, 不同评价指标下可能呈现不同结果。

从推理速度来看, 0.5 剪枝模型在两种输入尺寸下均具有最高 FPS。在 416×416 输入尺寸下, baseline 的 FPS 为 36.65, 0.3 剪枝模型提升至 43.01, 0.5 剪枝模型进一步提升至 50.94。在 640×640 输入尺寸下, baseline 的 FPS 为 20.34, 0.3 剪枝模型为 22.02, 0.5 剪枝模型提升至 34.96。由此可见, 剪枝能够提升模型推理速度, 且 0.5 剪枝在当前实验中速度提升更加明显。

为进一步展示不同剪枝比例下模型检测精度与推理速度之间的关系, 本文绘制剪枝模型精度—速度权衡对比图, 如图 7 所示。

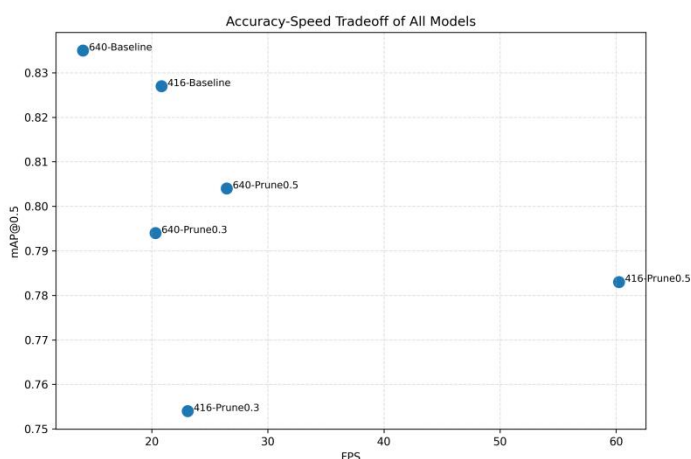


图 7 不同剪枝比例下模型精度—速度权衡对比

图 7 展示了 416×416 和 640×640 两种输入尺寸下 baseline、Pruned 0.3 和 Pruned 0.5 模型在 mAP@0.5 与 FPS 之间的关系。可以看出，剪枝模型整体向更高 FPS 方向移动，说明剪枝能够提升模型实时性。其中，Pruned 0.5 在两种输入尺寸下均

取得更高 FPS，同时 mAP@0.5 也高于 Pruned 0.3，说明在当前实验设置下，若以 mAP@0.5 和 FPS 作为主要评价指标，0.5 剪枝取得了更好的精度—速度折中。

3.5.4 剪枝比例对比小结

综合来看，0.3 和 0.5 剪枝均能够降低 YOLOv8n 模型参数量和计算量，其中 0.5 剪枝压缩效果更明显。在当前实验结果中，0.5 剪枝模型不仅参数量和 FLOPs 更低，而且 FPS 更高，说明较高剪枝比例能够有效提升模型部署效率。

但是，剪枝比例与检测精度之间并不是简单线性关系。通常情况下，剪枝比例越高，模型结构被压缩得越强，精度下降风险越大；但本实验中，0.5 剪枝模型的 mAP@0.5 反而高于 0.3 剪枝模型。这说明 0.5 剪枝并不一定比 0.3 剪枝精度更差，其实际效果还受到验证集规模、剪枝通道选择、微调恢复效果以及数据集类别特征等因素影响。

同时，剪枝比例与推理速度之间也不是严格线性关系。虽然 0.5 剪枝在本实验中取得了更高 FPS，但速度提升幅度并不完全等同于参数量或 FLOPs 的下降比例。实际推理速度还会受到模型结构规整程度、硬件环境、算子优化、数据读取、预处理和后处理等因素影响。

因此，本节实验表明，结构化剪枝能够有效降低 YOLOv8n 的模型复杂度，并提升推理速度；但剪枝比例、检测精度和实际速度之间并不存在简单线性关系。就当前结果而言，0.5 剪枝在参数量、FLOPs 和 FPS 方面均优于 0.3 剪枝，同

时 mAP@0.5 相较于 0.3 剪枝保持在更高水平, 因此在本组剪枝实验中表现出较好的压缩—精度—速度综合平衡。

3.6 全部模型综合对比

为进一步比较不同优化方法对 YOLOv8n 模型检测精度、模型复杂度和推理速度的影响, 本文对 baseline、知识蒸馏模型和剪枝模型进行统一评估。表 8 汇总了全部模型在 416×416 和 640×640 两种输入尺寸下的主要性能指标, 包括 mAP@0.5、mAP@0.5:0.95、FLOPs、参数量和 FPS。

表 8 全部模型统一评估结果汇总表

模型	输入尺寸	mAP@0.5	mAP@0.5:0.95	FLOPs(G)	Params(M)	FPS
YOLOv8n Baseline	416	0.827461	0.501727	1.709134	3.006818	36.652437
YOLOv8n Baseline	640	0.835223	0.497169	4.045286	3.006818	20.339714
Student Distilled	416	0.812	0.476853	1.709134	3.006818	27.301118
Student Distilled	640	0.808	0.497037	4.045286	3.006818	21.487929
Pruned 0.3	416	0.754252	0.435628	1.039015	1.682361	43.011956
Pruned 0.5	416	0.783491	0.440345	0.727646	1.058658	50.937156
Pruned 0.3	640	0.794406	0.483234	2.459208	1.682361	22.024683
Pruned 0.5	640	0.803914	0.451297	1.722240	1.058658	34.956740

注: 表 8 及后续综合对比采用统一评估脚本结果, 用于比较不同模型在相同评价口径下的相对性能差异。mAP@0.5 和 mAP@0.5:0.95 越高表示检测精度越好, FLOPs 和 Params 越低表示模型复杂度越低, FPS 越高表示推理速度越快。

从表 8 可以看出，不同优化方法对模型性能的影响存在明显差异。Baseline 模型在两种输入尺寸下均保持较高检测精度，其中 640×640 baseline 的 mAP@0.5 为 0.835，略高于 416×416 baseline 的 0.827；但 416×416 baseline 的 FPS 为 36.65，高于 640×640 baseline 的 20.34，说明较低输入尺寸具有更好的实时性。

知识蒸馏方法不会改变学生模型的结构和参数量。 416×416 与 640×640 蒸馏模型的 Params 均为 3.007M，FLOPs 也与对应 baseline 保持一致。从检测精度来看， 416×416 蒸馏模型的 mAP@0.5 为 0.784，低于对应 baseline；而 640×640 蒸馏模型的 mAP@0.5 为 0.836，与 640×640 baseline 的 0.835 基本持平并略高。说明蒸馏效果与输入分辨率和教师模型性能有关，并不一定在所有设置下都带来稳定提升。

剪枝方法能够显著降低模型复杂度。与 baseline 的 3.007M 参数量相比，Pruned 0.3 的参数量降低至 1.682M，Pruned 0.5 进一步降低至 1.059M。FLOPs 也随剪枝比例增加而明显下降，说明结构化剪枝能够有效减少模型计算量。从推理速度来看，Pruned 0.5 在 416×416 和 640×640 下的 FPS 分别达到 50.94 和 34.96，均高于对应 baseline 和 Pruned 0.3，体现出较好的加速效果。

从检测精度与速度的综合表现来看，若更关注检测精度， 640×640 baseline 和 640×640 蒸馏模型具有较高的 mAP@0.5，其中 640×640 蒸馏模型达到 0.836；若更关注轻量化部署和实时性， 416×416 Pruned 0.5 具有最低 FLOPs、最低参数量和最高 FPS，适合对推理速度要求较高的场景。 640×640 Pruned 0.5 则在保持较低参数量的同时取得 0.804 的 mAP@0.5 和 34.96 FPS，表现出较好的压缩—精度—速度平衡。

综合来看，知识蒸馏主要用于在不改变学生模型结构的条件下尝试改善检测精度，而剪枝主要用于降低模型参数量和计算量、提升推理速度。实验结果表明，不同优化方法各有侧重，模型选择应根据实际部署需求进行权衡：精度优先时可选择 640×640 baseline 或 640×640 蒸馏模型；实时性和轻量化优先时， 416×416 Pruned 0.5 更具优势；若需要在较高输入分辨率下兼顾检测效果与推理效率， 640×640 Pruned 0.5 是较合适的折中方案。

3.7 精度—速度权衡分析

为进一步直观比较不同模型在检测精度和推理速度之间的综合表现, 本文绘制不同模型在 416×416 和 640×640 输入尺寸下的精度—速度权衡对比图, 如图 8 所示。图中横轴表示推理速度 FPS, 数值越大表示推理速度越快; 纵轴表示 mAP@0.5, 数值越大表示检测精度越高。

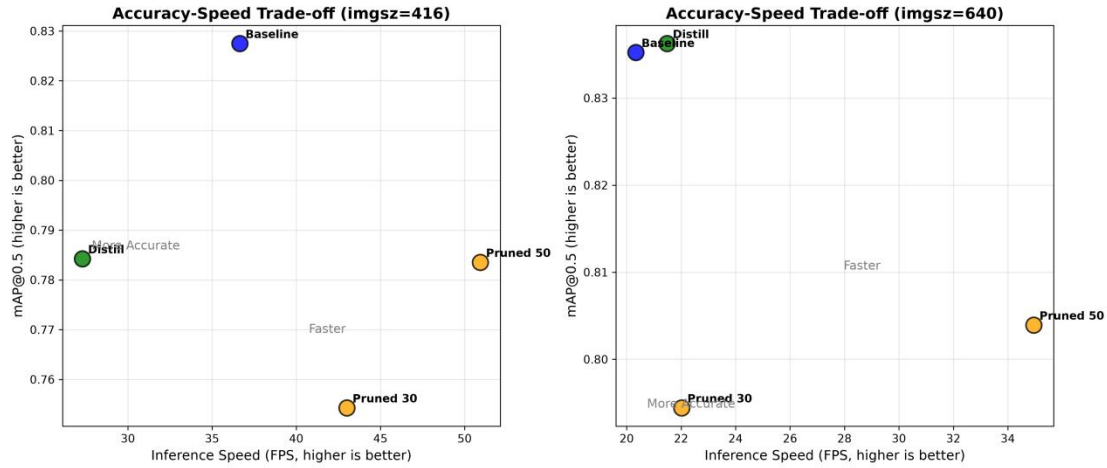


图 8 不同模型在两种输入尺寸下的精度—速度权衡对比

从图 8 可以看出, 在 416×416 输入尺寸下, baseline 模型具有最高的 mAP@0.5, 但其 FPS 低于 Pruned 0.5。Pruned 0.5 虽然 mAP@0.5 略低于 baseline, 但明显提升了推理速度, 说明在实时性优先的应用场景中, Pruned 0.5 具有更好的速度优势。相比之下, 416×416 蒸馏模型的 mAP@0.5 和 FPS 均未优于 baseline, 说明该组蒸馏结果在精度—速度权衡上表现一般。

在 640×640 输入尺寸下, 蒸馏模型的 mAP@0.5 与 baseline 基本接近并略高, 二者均处于较高精度区域; 但 Pruned 0.5 的 FPS 明显更高, 同时仍保持一定检测精度。因此, 640×640 Pruned 0.5 在精度略低于 baseline 和蒸馏模型的情况下, 获得了更明显的速度提升, 体现出较好的压缩部署价值。

综合两种输入尺寸可以发现, 不同模型并不存在单一最优结果, 而是对应不同应用需求。若以检测精度为优先目标, 640×640 baseline 或 640×640 蒸馏模型更合适; 若以实时性和部署效率为优先目标, 416×416 Pruned 0.5 具有更明显优势; 若希望在较高输入分辨率下兼顾精度和速度, 640×640 Pruned 0.5 是较合理的折中选择。

因此, 精度—速度权衡分析表明, 模型压缩并不是简单追求最高 mAP 或最

高 FPS，而是需要根据实际工业质检场景的需求，在检测精度、推理速度和模型复杂度之间进行综合选择。

3.8 混淆矩阵与类别误差分析

为进一步分析不同模型在各类钢材表面缺陷上的识别差异，本文选取 YOLOv8n Baseline-640（高分辨率基线模型）、Student Distilled-640（蒸馏最优模型）、Pruned 0.5-640（高分辨率剪枝综合较优模型）和 Pruned 0.5-416（速度最快的轻量化模型）四组代表性模型的混淆矩阵进行对比，如图 9 所示。

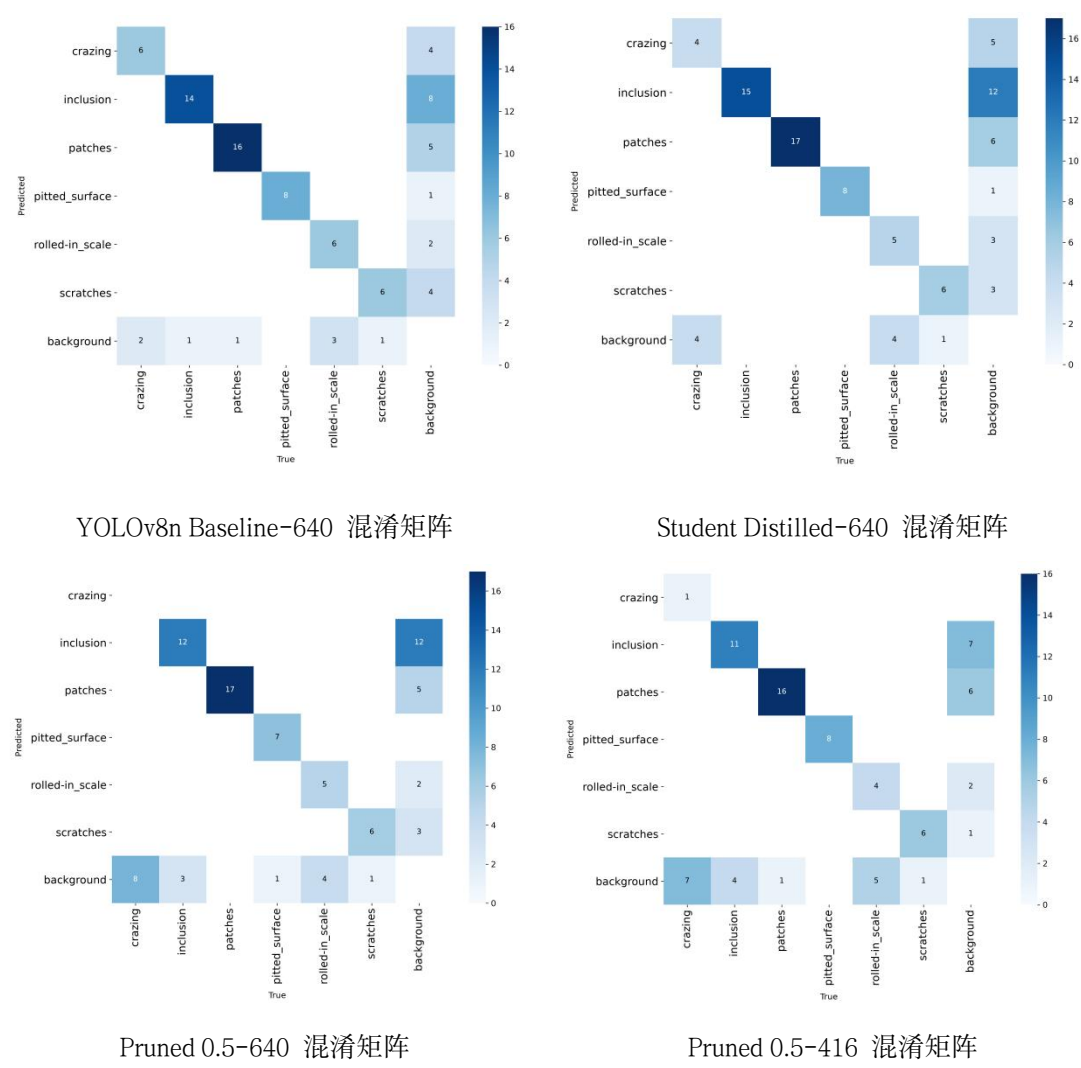


图 9 不同模型在验证集上的混淆矩阵对比

从图 9 可以看出，不同模型的主要错误并不是集中在缺陷类别之间的相互混淆，而是集中在缺陷目标与 background 之间的判断。也就是说，模型大多数情况下能够区分不同缺陷类别，但在部分样本中会将真实缺陷漏检为背景，或将背

景区域误检为某类缺陷。因此，漏检和背景误检是当前模型类别误差的主要来源。

从各类别表现来看，patches、pitted_surface 和 scratches 在不同模型中整体识别较稳定。其中，patches 在多组模型中均保持较高的正确识别数量，说明其块状区域特征较明显，模型较容易学习；pitted_surface 和 scratches 也能被较稳定检测，说明这两类缺陷的形态特征相对清晰。

相比之下，crazing 和 rolled-in_scale 是更易出现错误的类别。crazing 通常表现为细碎裂纹，纹理弱、边界不明显，容易被模型判断为 background；rolled-in_scale 的形态不规则，且与钢材表面背景纹理相似，也容易产生漏检。该现象与前文各模型 mAP 指标中 crazing 和 rolled-in_scale 表现相对较弱的结果一致。

对比不同模型可以发现，Student Distilled-640 虽然在部分类别上具有较高的预测准确性，但其对部分缺陷目标的召回能力不足，容易出现漏检，这也解释了其 Precision 提升但 Recall 下降的现象。Pruned 0.5 模型在压缩参数量和提升推理速度的同时，部分弱纹理类别的识别稳定性受到影响，说明剪枝可能削弱部分细粒度特征表达能力。

综合来看，混淆矩阵结果表明，模型压缩后整体类别区分能力仍然存在，但弱纹理、小目标和边界模糊类别仍是主要难点。后续优化应重点关注 crazing 和 rolled-in_scale 等易漏检类别，可通过调整置信度阈值、增强弱纹理样本训练、优化蒸馏损失权重或在剪枝后加强微调来改善模型对难检缺陷的识别能力。

3.9 检测可视化与误检漏检案例分析

为进一步分析模型在实际检测过程中的错误类型，本文选取典型验证样本进行可视化展示，如图 10 所示。该图主要反映模型在弱纹理、低对比度和边界模糊缺陷上的检测情况。

从图 10 可以看出，crazing 类缺陷在灰度图像中通常表现为细碎裂纹，纹理较弱，且与钢材背景纹理相似，因此模型容易出现漏检或低置信度预测。样本 1 和样本 3 中虽然存在疑似裂纹纹理，但图中未显示明显预测框，说明模型对该类弱纹理缺陷的检出能力仍不稳定。样本 2 中模型检测出了 crazing 区域，但置信度仅为 0.33，说明模型虽然能够捕捉到部分裂纹特征，但对该目标的判断并不

充分。

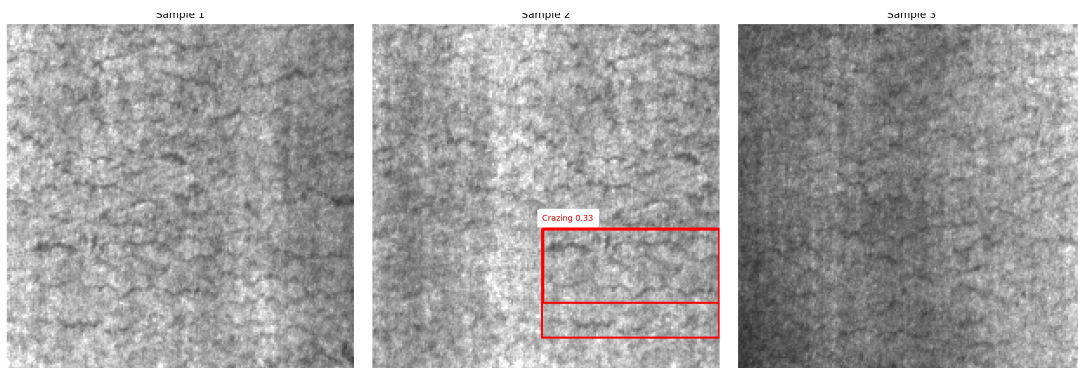


图 10 检测可视化与误检漏检案例

该现象与前文混淆矩阵分析结果一致：模型的主要错误并非缺陷类别之间的大量混淆，而是弱纹理缺陷与 background 之间的判断不稳定。对于 crazing 这类边界模糊、纹理细碎的缺陷，模型容易将其视为背景，从而产生漏检；即使能够检测到目标，也可能出现置信度偏低的问题。

综合来看，检测可视化结果进一步说明，当前模型对 patches、pitted_surface 等特征明显的缺陷具有较好检测能力，但对 crazing 等弱纹理缺陷仍存在不足。后续优化可重点从增加弱纹理样本增强、调整置信度阈值、改进蒸馏损失或在剪枝后加强微调等方面入手，以提升模型对难检缺陷的识别稳定性。

4 讨论

4.1 不同压缩方法的优缺点

知识蒸馏和结构化剪枝对模型的作用机制不同。知识蒸馏不改变 YOLOv8n 学生模型的网络结构和参数量，主要通过教师模型提供监督信息来改善学生模型的特征学习能力。因此，蒸馏方法适合在模型结构保持不变的前提下尝试提升检测性能。但其缺点是训练流程更复杂，需要额外的教师模型，并且蒸馏效果依赖于教师模型质量和蒸馏策略设置，并不一定稳定提升所有指标。

结构化剪枝则直接减少模型冗余通道，能够降低参数量和 FLOPs，并提升推理速度，更适合轻量化部署需求。但剪枝会改变模型结构，可能削弱部分特征表达能力，导致检测精度下降，尤其对弱纹理、小目标或边界模糊类别影响更明显。因此，剪枝后需要通过微调恢复模型性能。

总体来看，知识蒸馏更偏向精度优化，结构化剪枝更偏向模型压缩和速度提

升，二者适用于不同优化目标。

4.2 输入分辨率对部署的影响

输入分辨率直接影响检测精度和推理速度。 416×416 输入尺寸计算开销较低，推理速度更快，更适合实时性优先的工业检测和边缘端部署场景。 640×640 输入尺寸能够保留更多图像细节，更适合精度优先或缺陷纹理较细的检测场景。

因此， 416×416 和 640×640 并不是简单的优劣关系，而是对应不同应用需求。实际部署时，应根据生产线速度要求、硬件算力和允许的漏检风险选择合适输入尺寸。

4.3 实际部署限制

模型压缩后的理论 FLOPs 降低并不一定完全等价于实际 FPS 同比例提升。真实推理速度还会受到推理框架、硬件平台、算子优化、内存访问、数据读取、预处理和后处理等因素影响。

因此，FLOPs、参数量和 FPS 应结合分析。FLOPs 和参数量用于衡量模型复杂度，FPS 用于反映具体环境下的实际推理效率。本文实验结果更适合用于同一实验环境下不同模型之间的相对比较，实际部署时仍需在目标硬件和推理框架上重新测试。

综合来看，模型优化不能只追求最高精度或最快速度，而应在检测精度、模型规模、推理速度和部署稳定性之间取得平衡。

5 结论

本文面向工业钢材表面缺陷检测场景，基于 NEU-DET 数据集构建了以 YOLOv8n 为核心的轻量级目标检测实验流程，并围绕输入分辨率、知识蒸馏和结构化剪枝等方面开展了模型优化与对比分析。

实验结果表明，YOLOv8n baseline 能够在较低参数量条件下完成钢材表面缺陷检测任务，具备作为轻量化优化基础模型的可行性。不同输入分辨率对模型性能具有明显影响： 416×416 输入尺寸推理速度更快，更适合实时性优先场景；

640 × 640 输入尺寸能够保留更多图像细节，更适合精度优先场景，二者体现出典型的精度—速度权衡关系。

在模型压缩方法方面，知识蒸馏能够在不改变学生模型结构和参数量的前提下尝试改善模型检测能力，但其效果受教师模型性能、输入分辨率和蒸馏策略影响，并不必然带来所有指标的同步提升。结构化剪枝能够显著降低模型参数量和计算量，并提升推理速度，但同时可能造成一定检测精度损失，需要通过微调恢复模型性能。

综合来看，本文实验验证了模型压缩方法在工业缺陷检测轻量化部署中的应用价值。通过合理选择输入分辨率、蒸馏策略和剪枝比例，可以在检测精度、模型规模和推理速度之间取得更合适的平衡，从而提升 YOLOv8n 在边缘工业质检场景中的部署潜力。

6 分工声明

- 1、刘宇欣：环境配置、下载 NEU-DET、训练 YOLOv8n baseline，保存模型。
- 2、刘紫涵：使用 YOLOv8m 作为教师蒸馏 YOLOv8n，分别对 416 和 640 做蒸馏实验。
- 3、余玉媛：对 baseline 模型进行通道剪枝，并进行重新训练和微调。
- 4、王芯：评估及可视化，计算所有模型的 mAP/FLOPs/参数量/FPS，画图。
- 5、李钰：统稿报告、做 PPT、准备汇报答辩。